گیت چیست؟

گیت (Git) یک سیستم کنترل نسخهٔ توزیع‌شده (Distributed Version Control System) است. برای درک بهتر، تصور کنید در حال نوشتن یک کتاب هستید و می‌خواهید تمام تغییرات متن را ذخیره کنید تا اگر روزی از یک بخش پشیمان شدید، به نسخهٔ قبلی برگردید. گیت دقیقاً همین کار را برای پروژه‌های برنامه‌نویسی انجام می‌دهد.

در گیت، هر بار که بخشی از کارتان به یک نقطهٔ قابل‌قبول می‌رسد — مثلاً یک باگ را برطرف کرده‌اید یا یک ویژگی جدید اضافه کرده‌اید — یک **نسخهٔ کامل از کل پروژه در آن لحظه** ذخیره می‌شود. به این ذخیره‌سازی **commit** می‌گویند.

مهم‌ترین توانایی‌های گیت عبارت‌اند از:

- بازگشت به هر نسخهٔ قبلی (هر commit)
- مشاهدهٔ تاریخچهٔ کامل تغییرات (چه کسی، چه زمانی، چه چیزی را تغییر داده)
- همکاری هم‌زمان چند نفر روی یک پروژه، بدون ترس از خراب شدن کد
- ادغام (merge) تغییرات افراد مختلف به‌صورت هوشمند و خودکار

مخزن (Repository) چیست؟

مخزن (Repository یا به‌اختصار repo) همان پوشه‌ای است که گیت مدیریت آن را بر عهده می‌گیرد. وقتی یک پوشه را به گیت می‌سپارید، گیت یک زیرپوشهٔ مخفی به نام `.git` در آن ایجاد می‌کند. تمام اطلاعات نسخه‌ها، commitها، شاخه‌ها و تنظیمات پروژه در همین `.git` نگهداری می‌شود.

مخزن می‌تواند دو حالت داشته باشد:

- **محلی (local):** روی رایانهٔ خودتان
- **ریموت (remote):** روی یک سرور یا سرویس اینترنتی مانند گیت‌هاب

شما معمولاً روی مخزن محلی کار می‌کنید و سپس تغییرات را با مخزن ریموت هم‌گام می‌کنید.

گیت‌هاب (GitHub) چیست؟

گیت‌هاب یک سرویس میزبانی مخازن گیت روی اینترنت است.
گیت ابزاری است که روی رایانهٔ شما نسخه‌ها را مدیریت می‌کند، اما گیت‌هاب محلی برای نگهداری آن مخازن روی ابر (Cloud) فراهم می‌آورد و ابزارهای پیشرفته‌ای برای همکاری تیمی به آن می‌افزاید:

- اشتراک‌گذاری کد با دیگران
- درخواست ادغام تغییرات (**Pull Request**) و امکان بررسی کد (**Code Review**)
- مدیریت وظایف و گزارش باگ‌ها (**Issues**)
- میزبانی وب‌سایت‌های ساده و استاتیک (**GitHub Pages**)
- مستندسازی خودکار و ابزارهای مدیریت پروژه

به زبان ساده: گیت مغز عملیات کنترل نسخه است و گیت‌هاب فضایی برای اشتراک و همکاری روی آن مخازن.

بررسی نصب گیت

ابتدا باید مطمئن شوید که گیت روی سیستم‌تان نصب است. ترمینال (Command Prompt، PowerShell یا Terminal) را باز کنید و دستور زیر را بنویسید:

```bash
git --version
```

اگر شماره‌ای مانند `git version 2.44.0` نمایش داده شد، گیت آمادهٔ کار است.
در غیر این‌صورت، از git-scm.com نسخهٔ مناسب سیستم‌عامل خود را دانلود و نصب کنید.

پیکربندی اولیه (تنها یک بار)

پیش از شروع کار با گیت، لازم است نام و ایمیل خود را به آن معرفی کنید. این اطلاعات کنار هر commit ثبت می‌شود تا مشخص شود چه کسی مسئول آن تغییر بوده است.

```bash
git config --global user.name "نام شما"
git config --global user.email "ایمیل شما"
```

شروع کار: ساختن یک مخزن گیت

برای اینکه گیت یک پوشه را تحت نظر بگیرد، وارد آن پوشه شوید و دستور زیر را اجرا کنید:

```bash
git init
```

این دستور یک مخزن خالی گیت در پوشهٔ فعلی ایجاد می‌کند (با ساختن پوشهٔ مخفی `.git`).
از این لحظه، گیت آمادهٔ ردیابی تغییرات این پوشه است.

اگر یک پروژهٔ موجود را از گیت‌هاب دریافت می‌کنید، نیازی به `git init` ندارید؛ به‌جای آن از `git clone` استفاده می‌کنید که جلوتر توضیح داده خواهد شد.

سه مرحلهٔ اصلی در گیت

گیت یک روند کاری سه‌مرحله‌ای دارد که فهم آن، کلید یادگیری گیت است:

1. **دایرکتوری کاری (Working Directory):**
   همان فایل‌ها و پوشه‌های معمولی پروژه. هر تغییری انجام می‌دهید (ویرایش، ایجاد یا حذف فایل) در این بخش اتفاق می‌افتد.

2. **محوطهٔ آماده‌سازی (Staging Area) یا Index:**
   جایی که فایل‌هایی را که می‌خواهید در commit بعدی ثبت شوند، انتخاب می‌کنید. این کار مانند بسته‌بندی اقلام پیش از ارسال است.
   این مرحله به شما اجازه می‌دهد همهٔ تغییرات را یک‌جا commit نکنید و فقط بخش‌های مشخصی را ثبت کنید.

3. **مخزن (Repository):**
   جایی که commitها ذخیره می‌شوند و تاریخچهٔ دائمی پروژه را شکل می‌دهند. پس از commit، نسخهٔ جدید به این بخش اضافه می‌شود.

ثبت تغییرات: add و commit

برای ذخیرهٔ یک تغییر در تاریخچهٔ گیت، باید دو گام را پشت سر بگذارید:

گام اول: افزودن به محوطهٔ آماده‌سازی (stage کردن)

با دستور `git add` فایل‌هایی را که می‌خواهید در commit بعدی قرار گیرند، انتخاب می‌کنید:

```bash
git add نام_فایل
# یا برای افزودن همهٔ فایل‌های تغییریافته و جدید:
git add .
```

گام دوم: ساختن commit

اکنون تغییرات انتخاب‌شده را برای همیشه در تاریخچه ثبت کنید:

```bash
git commit -m "توضیح کوتاه دربارهٔ تغییرات"
```

پارامتر `m-` مخفف message (پیام) است و متنی که می‌نویسید باید خلاصهٔ گویایی از کاری که انجام داده‌اید باشد.

مثال:

```bash
git commit -m "اضافه کردن فرم ورود کاربر"
```

پس از این دستور، یک commit جدید با یک شناسهٔ یکتا ساخته می‌شود و بعداً می‌توانید به آن مراجعه کنید.

بررسی وضعیت و مشاهدهٔ تاریخچه

دیدن وضعیت فعلی

این دستور نشان می‌دهد کدام فایل‌ها تغییر کرده‌اند، کدام‌ها برای commit نشانه‌گذاری (stage) شده‌اند و کدام‌ها هنوز پیگیری نمی‌شوند:

```bash
git status
```

مشاهدهٔ تفاوت‌ها پیش از stage

برای دیدن اینکه چه خطوطی دقیقاً تغییر کرده‌اند (و هنوز stage نشده‌اند):

```bash
git diff
```

مشاهدهٔ تاریخچهٔ commitها

```bash
git log
```

این فرمان لیست commitها را به‌ترتیب از جدیدترین به قدیمی‌ترین نشان می‌دهد.
برای دیدن یک نسخهٔ خلاصه‌تر (فقط شناسه و پیام):

```bash
git log --oneline
```

برای خارج شدن از محیط log، کلید `q` را بزنید.

کار با شاخه‌ها (Branch)

شاخه (Branch) یکی از قدرتمندترین قابلیت‌های گیت است.
تصور کنید می‌خواهید یک قابلیت جدید را امتحان کنید، اما نمی‌خواهید کد اصلی (نسخهٔ پایدار) خراب شود. یک شاخهٔ جداگانه می‌سازید، تغییرات را در آن انجام می‌دهید، و اگر همه چیز خوب پیش رفت، در نهایت آن شاخه را با کد اصلی ادغام (merge) می‌کنید.

ساختن شاخهٔ جدید

```bash
git branch نام_شاخه
```

این فرمان فقط شاخه را می‌سازد، اما هنوز روی همان شاخهٔ قبلی هستید.

رفتن به شاخه

```bash
git switch نام_شاخه
# روش قدیمی‌تر (هنوز پرکاربرد):
git checkout نام_شاخه
```

ساختن و رفتن هم‌زمان به شاخهٔ جدید

```bash
git switch -c نام_شاخه
```

از این پس، هر commitای که انجام دهید روی این شاخه ثبت می‌شود و شاخهٔ اصلی دست‌نخورده باقی می‌ماند.

ادغام یک شاخه با شاخهٔ فعلی (Merge)

وقتی کارتان روی شاخهٔ فرعی تمام شد، به شاخهٔ اصلی برگردید و شاخهٔ فرعی را در آن ادغام کنید:

```bash
git switch main
git merge نام_شاخه
```

اگر گیت بتواند تغییرات را به‌صورت خودکار ترکیب کند، ادغام بدون مشکل انجام می‌شود.
در غیر این‌صورت یک **تداخل (Conflict)** رخ می‌دهد که از شما می‌خواهد تصمیم بگیرید کدام تغییر باقی بماند. (نگران نباشید؛ با تمرین، مدیریت تداخل‌ها را یاد خواهید گرفت.)

دیدن لیست همهٔ شاخه‌ها

```bash
git branch
```

شاخه‌ای که در آن هستید با علامت ستاره (`*`) مشخص می‌شود.

اتصال به گیت‌هاب و کار با مخازن ریموت

برای اشتراک‌گذاری پروژه یا دریافت پروژه‌ای که دیگران روی آن کار کرده‌اند، باید با مخازن ریموت ارتباط برقرار کنید.

دریافت یک کپی کامل از مخزن ریموت (clone)

این فرمان یک پوشه می‌سازد، کل مخزن (با تمام commitها و شاخه‌ها) را دانلود می‌کند و ارتباط با سرور اصلی را به‌صورت خودکار تنظیم می‌کند:

```bash
git clone آدرس_مخزن
```

آدرس مخزن را معمولاً از گیت‌هاب (دکمهٔ Code) کپی می‌کنید؛ چیزی شبیه `https://github.com/user/repo.git` .
پس از clone، یک پوشه با نام مخزن خواهید داشت و می‌توانید بلافاصله کار را شروع کنید.

ارسال commitها به ریموت (push)

وقتی روی مخزن محلی خود commitهایی ساخته‌اید و می‌خواهید آن‌ها را روی گیت‌هاب یا سرور دیگری به اشتراک بگذارید:

```bash
git push origin main
```

- `origin` : نام پیش‌فرضی است که گیت به مخزن ریموت اختصاص می‌دهد (برای دیدن لیست ریموت‌ها می‌توانید از `git remote -v` استفاده کنید).
- `main` : نام شاخه‌ای که می‌خواهید روی سرور به‌روز شود (در پروژه‌های قدیمی‌تر ممکن است `master` باشد).

دریافت آخرین تغییرات از ریموت (pull)

اگر دیگران روی مخزن ریموت commit جدیدی فرستاده‌اند و شما می‌خواهید آن‌ها را دریافت و با کارتان ادغام کنید:

```bash
git pull
```

این فرمان هم دانلود و هم ادغام را انجام می‌دهد. معمولاً اولین کاری که صبح انجام می‌دهید، `git pull` است تا آخرین تغییرات تیم را داشته باشید.

نادیده گرفتن فایل‌ها با `.gitignore`

برخی فایل‌ها نباید در مخزن گیت ثبت شوند: فایل‌های موقتی، پوشه‌های وابستگی‌ها، یا اطلاعات حساس (پسورد، کلید).
برای این منظور، فایلی به نام `.gitignore` در ریشهٔ پروژه بسازید و نام فایل‌ها یا پوشه‌هایی را که می‌خواهید نادیده گرفته شوند، داخل آن بنویسید.

مثال ساده از محتوای `.gitignore` :

```text
node_modules/
.env
*.log
```

از آن پس، گیت این موارد را نه در `status` نشان می‌دهد و نه با `. add` به محوطهٔ آماده‌سازی می‌افزاید.